Fairy-Stockfish for our Blobfish purposes

by Emily Tew <3

There are basically no official docs for the Python side, so most of this was figured out by just poking at it. All the examples below were run and the outputs are real for ref

what it is and why we use it

Stockfish is a very good chess engine. Fairy-Stockfish is a version of it that knows weird chess variants, including the two small boards we shall use. Regular Stockfish only plays normal 8x8 chess, which is why we need the fairy one.

We use it for these two jobs:

1. the referee (pyffish, a python library) which governs legal moves, board positions, is the game over
2. the opponent (the actual fairy-stockfish engine, a separate program) which plays against Blobfish so we can see how good it is

The referee never plays or thinks, it just knows rules. The opponent is the actual bot

Board sizes and their variant names are thusly:

- 5x5 = "gardner"
- 6x6 = "losalamos" (this one has no bishops)
- 8x8 = "chess"

I also put these in `blobfish.encoding.VARIANTS`, so the rest of our project can just say `size=5` for the sake of ease huzzah

the ref(eree) (pyffish)

I put it in our package, so `pip install -e .` from the project folder installs it easy peez I think it may include compiled C++ code though, so if you happen to see any issue with that alert the gang

This is the one weird thing: there is no board object. Every function takes the same three things, the variant name, the starting FEN, and the list of moves played so far. pyffish works out the current position from those so making a move just means adding it to the list. look at this:

```
import pyffish as sf

game = "gardner"
start = sf.start_fen(game)          # "rnbqk/ppppp/5/PPPPP/RNBQK w - - 0 1"
moves = []

sf.legal_moves(game, start, moves)  # ['a2a3', 'b2b3', 'c2c3', 'd2d3', 'e2e3',
'b1a3', 'b1c3']

moves.append("a2a3")                # make a move

sf.get_fen(game, start, moves)      # "rnbqk/ppppp/P4/1PPPP/RNBQK b - - 0 1"
```

Moves are written from square then to square like "a2a3". Any promotions add a letter for the new piece, so "a4a5q" is a queen (also "a4a5r", "a4a5b", "a4a5n"). `sf.get_san(game, start, "a2a3")` gives the readable name ("a3") if you want to print games

is the game over or what

this is funky - pyffish does NOT end the game for you. this is not ideal, but self-play has to check both of these after every move:

```
# 1. no legal moves means checkmate or stalemate
if not sf.legal_moves(game, start, moves):
    result = sf.game_result(game, start, moves)
    # -32000 = the player whose turn it is got checkmated (loss)
    #      0 = stalemate (draw)

# 2. draws by rule (50 move rule, only kings left, repetition, etc)
over, result = sf.is_optional_game_end(game, start, moves)
# over is True if the game should end, result is 0 for a draw
```

Results are always from the point of view of whoever's turn it is. Our value scale is -1 / 0 / +1, so convert with:

```
value = result / sf.VALUE_MATE    # sf.VALUE_MATE is 32000, sf.VALUE_DRAW is 0
```

using it with the encoding i did

`sf.get_fen(...)` gives the FEN that goes straight into `encode()`:

```
from blobfish.encoding import encode

x = encode(sf.get_fen(game, start, moves))    # tensor of shape (18, 5, 5)
```

Remember THE BIG FLIP: on black's turn `encode()` turns the board upside down so the network always sees it from the side that's moving duh. pyffish always uses the real squares though so when the network picks a move on blacks turn, the move has to be flipped back before giving it to pyffish or we'll be playing moves on the wrong side of the board (bad)

On a 5x5 board rank r flips to rank 6 - r (so a5a4 and a1a2 are the same move from each side's view) This gets handled by the move to index mapping

other stuff that i discovered

```
sf.gives_check(game, start, moves)            # is the side to move in check? yes
or no
sf.is_capture(game, start, moves, "a2a3")     # does this move capture something?
yes or no
sf.has_insufficient_material(game, start, moves)   # (white can't win, black can't
win)
sf.validate_fen(fen, game)                    # 1 if the fen is valid
sf.variants()                                 # every variant pyffish knows (there
are 121 and some arent even chess)
```

here is the bad news

pyffish is much slower than python-chess...every call takes about 1.7 ms no matter the board size or how long the game is. python-chess does the same thing in about 0.015 ms yeeeouch

MCTS sims will need needs a few calls, so this could make self-play slower

My thoughts are this, we could:

- call it as little as possible, like caching the legal moves for each position in the MCTS tree or something
- run several games at once on different CPU cores, like we discussed

- for gardner, writing our own small move generator is an option since gardner has no castling or en passant

I am thinking of joining the Fairy-Stockfish Discord to see if anyboyd would answer my questions

also.... pass the full move list, not just the current FEN. Calling it with only the current FEN and an empty move list works, but then it can't detect repetition draws since a FEN has no history

the opponent (later)

Getting the engine:

- download page is this, behold https://fairy-stockfish.github.io/download/
- prebuilt versions are Windows and Linux only so for my lowly Mac it has to be built from source
- use the normal build we dont need largeboard

the engine is a separate program that speaks UCI, which is a plain text protocol that chess engines use You start it, send it lines of text and read what it says back. Something like this perhaps:

```
uci                                          → engine lists its options
setoption name UCI_Variant value gardner     → play 5x5
position fen rnbqk/ppppp/5/PPPPP/RNBQK w - - 0 1 moves a2a3
go depth 5                                   → engine thinks
                                             ← bestmove ... blablbalh
```

- for 8x8, python-chess's `chess.engine` module handles all of this for us
- for 5x5 and 6x6, python-chess can't represent the board, so we talk UCI directly by starting the program using `subprocess` and sending those lines
- lower search depth makes it weaker so Stockfish lets u choose strength
- stockfish has no opening book so it plays the same way every time at a given setting so we need to start test games from a few different positions

Fairyground: a website that shows a board and supports almost every variant Fairy-Stockfish knows. Good for pasting in a FEN from self-play and checking that the moves make sense.

where to find more

- when in doubt, `dir(sf)` lists every function, `help(sf.some_function)` shows any built-in help
- this doc is honestly the most complete pyffish documentation out there right now
- projects that use pyffish (linked from the wiki's API page), like pychess.org, are like the only examples of real usage